Loops and Modules in JavaScript

Part 1: Iteration in JavaScript

Traditional for Loop

The traditional for loop is the most basic way to iterate over arrays. It gives you complete control over the iteration process.

```
const arr = [3, 5, 1, 3, 6];

for(let i = 0; i < arr.length; i++){
    console.log(arr[i]);
}
```

Output:

3 5 1 3 6

for...of Loop (ES6)

The `for...of` loop is a modern way to iterate over iterable objects like arrays, strings, Maps, and Sets. It directly gives you the values without needing an index.

```
const arr = [3, 5, 1, 3, 6];

for(let num of arr){
    console.log(num);
}
```

Output:

3 5 1 3 6

Powered by SPRK Technologies

for...in Loop

The `for...in` loop iterates over the enumerable properties (indexes/keys) of an object. When used with arrays, it returns the indexes as strings.

```
const arr = [3, 5, 1, 3, 6];

for(let idx in arr){
    console.log(idx);
}
```

Output:

0 1 2 3 4

Iterating Over Objects

Objects Are Not Directly Iterable

Unlike arrays, objects cannot be used directly with `for...of` loops. Attempting to do so will result in an error because objects are not iterable.

```
const user = {
    name: "Krunal",
    age: 30,
    salary: 837893
};

// This will throw an error:
// for(let data of user){ ... }
```

Output:

TypeError: user is not iterable

Powered by SPRK Technologies

Using Object.entries() with for...of

To iterate over an object's key-value pairs using `for...of`, convert the object to an array using `Object.entries()`. This method returns an array of [key, value] pairs.

```
const user = {
    name: "Krunal",
    age: 30,
    salary: 837893
};

for(let data of Object.entries(user)){
    console.log(data);
}
```

Output:

['name', 'Krunal'] ['age', 30] ['salary', 837893]

Using for...in with Objects

The `for...in` loop works directly with objects and returns the property names (keys).

```
const user = {
    name: "Krunal",
    age: 30,
    salary: 837893
};

for(let data in user){
    console.log(data);
}
```

Output:

name age salary

Powered by SPRK Technologies

To access the values while using `for...in`, use bracket notation:

```
for(let key in user){
    console.log(key + ": " + user[key]);
}
```

Output:

name: Krunal age: 30 salary: 837893

Choosing the Right Loop

- **for...of:** Use when you need to iterate over values of arrays, strings, Maps, or Sets
- **for...in:** Use when you need to iterate over object keys or array indexes
- **Traditional for:** Use when you need precise control over iteration or need the index

---

Part 2: JavaScript Modules

Introduction to Modules

JavaScript modules allow you to split your code into separate files. Each file can export functions, objects, or values, and other files can import them. This makes code more organized, reusable, and easier to maintain.

Exporting from a Module

There are two types of exports in JavaScript: named exports and default exports.

Named Exports

Named exports allow you to export multiple values from a single file. Each export must be imported using the exact same name.

File: mymath.js

Powered by SPRK Technologies

```
export const PI = 3.14;

export function area(r = 0){
    return PI * r * r;
}
```

> **Note:**
> The `export` keyword makes the variable or function available to other files. Multiple named exports can exist in one file.

Default Export

A default export is the main value exported from a module. Each module can have only one default export.

```
export default function greet(){
    console.log("Hello");
}
```

> **Note:**
> Default exports are useful when a module primarily exports one thing, like a class or a main function.

Importing from a Module

To use exported values in another file, you need to import them.

Named Imports

Named imports must use the exact names that were exported, and they must be enclosed in curly braces.

File: demo.js

Powered by SPRK Technologies

```
import { PI, area } from "./mymath";

console.log(area(5));
```

Output:

78.5

Default Import

Default imports can use any name you choose. They do not require curly braces.

```
import greet from "./mymath";

greet();
```

Output:

Hello

Combining Named and Default Imports

You can import both named and default exports in a single statement.

```
import greet, { PI, area } from "./mymath";

console.log(area(5));
greet();
```

Output:

78.5 Hello

Powered by SPRK Technologies

Import Aliases

You can rename imports using the `as` keyword to avoid naming conflicts.

```
import { area as calculateArea } from "./mymath";

console.log(calculateArea(5));
```

Output:

78.5

Importing Everything

You can import all named exports from a module using the asterisk (*) syntax.

```
import * as math from "./mymath";

console.log(math.PI);
console.log(math.area(5));
```

Output:

3.14 78.5

Module Best Practices

- Use named exports when exporting multiple related items
- Use default exports for the primary functionality of a module
- Keep modules focused on a single responsibility
- Use descriptive file names that reflect the module's purpose
- Avoid circular dependencies between modules

Powered by SPRK Technologies